Deliver AI with Less Compute

Architecture & Build Plan — a measured efficiency layer for LLM workloads

Version 1.0 · 30 October 2026 · Owner: Hamzah · Submission deadline: Saturday 31 October, 20:00 (Asia/Riyadh) · Companion files: **CLAUDE.md** (build rules) and **docs/SPEC.md** (full specification)

In one paragraph. We are building a layer that sits in front of any AI workload and answers each request with the cheapest *model + thinking-effort* combination that still meets that task's accuracy and speed targets — and we prove it on data the system has never seen. The showcase is a fictional, bilingual (English/Arabic) Saudi motor-insurance claims desk. The headline number is **cost per successful task**: every dollar spent, including retries, checks and failed attempts, divided by the answers that were both correct and on time.

1. The problem and the thesis

Organisations want AI results with less compute, but cheap only counts if the answer is right and on time. Three facts from current (October 2026) pricing shape the whole design:

Model	Input $ / M tokens	Output $ / M tokens	Cache read	Relative price
Claude Haiku 5.5	0.10	0.50	0.01 (0.1×)	1
Claude Sonnet 5.5	2.00	10.00	0.10 (0.05×)	≈ 20
Claude Opus 5.5	4.00	20.00	0.20 (0.05×)	≈ 40

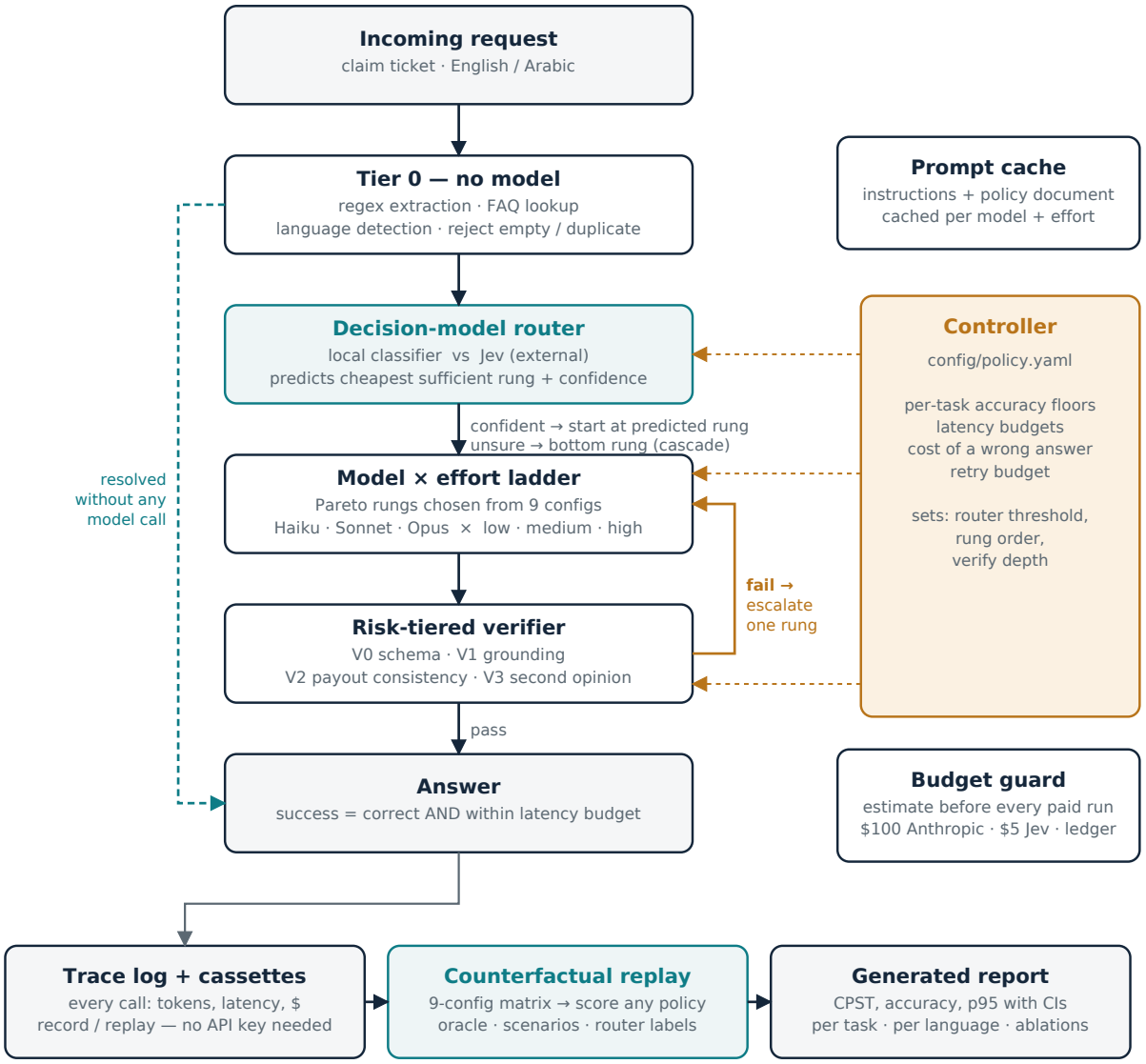
Prices from consistent third-party pricing pages, 29–30 Oct 2026; re-verified against the official page in Phase 0. Haiku rates apply to prompts up to 100K tokens.

- **Trying the cheap model first is now almost free.** A wasted Haiku attempt costs about 2% of a Sonnet call, so the money is not saved by skipping the junior model.
- **The expensive mistakes happen higher up.** Failing at Sonnet-medium before succeeding at Sonnet-high wastes roughly half the final cost. Choosing the right rung among the expensive configurations is where savings live.
- **Effort is the biggest lever.** With caching, input on Sonnet/Opus costs 5% of list price, so output and thinking tokens dominate the bill — and the effort setting controls exactly those.

Thesis. The decision model's job is not to avoid the cheap call. It is to pick the right (model, effort) rung among the expensive ones, and to skip the waiting on hard requests.

2. System architecture

Each request flows top to bottom. Easy requests should end at Tier 0 or the first rung; hard ones climb the ladder only as far as they need to. Every call is recorded, which makes all evaluation replayable without spending again.



Component	What it does (plain terms)	Technical detail
Tier 0	Answers what needs no AI: exact FAQ matches, unambiguous ID extraction.	Deterministic rules; steps aside on any ambiguity.
Router	Guesses how hard a request is before any model runs.	Local TF-IDF + logistic regression vs TypeSafe Jev (zero-shot). Confidence threshold; low confidence → cascade.
Ladder	The ordered list of model + effort options worth using for each task.	Pareto-efficient subset of 9 configs; dominated configs removed using dev data.
Verifier	Checks answers hardest where a mistake is expensive.	V0 schema · V1 grounding · V2 payout recomputation · V3 second opinion when expected loss > check cost.
Controller	Applies the business rules for speed, accuracy and risk.	config/policy.yaml: per-task floors, latency budgets, cost of a wrong answer, retry budget.
Cache layout	Re-uses the unchanging start of every prompt at a discount.	One static system block (instructions + policy doc); ticket in the user turn; 5-minute TTL.
Record / replay	Keeps a receipt of every call so results can be reproduced for free.	Cassettes keyed by request hash; replay mode needs no API key.

3. The model × effort ladder and the router contest

Three models at three effort levels give nine configurations. Phase 0 measures every one of them (acceptance, real token use, latency, cache behaviour) before the design relies on it.

	low effort	medium effort	high effort
Haiku 5.5	haiku-low	haiku-medium	haiku-high
Sonnet 5.5	sonnet-low	sonnet-medium	sonnet-high
Opus 5.5	opus-low	opus-medium	opus-high

- **Dominated configurations are dropped.** If one config is both more expensive and less accurate than another on dev data, it never appears on the ladder. Opus is only 2× Sonnet, so Opus-low may beat Sonnet-high.
- **Escalation is one rung at a time**, and the verifier decides when to climb.
- **max and xhigh effort are excluded** because they exist only on some models.

Three ways to choose the starting rung — decided by evidence, not preference

Contender	Strength	Weakness
Cascade only	Needs no training; reacts to the actual answer.	Hard requests wait through failed cheap attempts.
Local classifier	Free, sub-millisecond, learns our models' real failure patterns.	Needs recorded outcomes; can over-fit a small dataset (cross-validated to guard against it).
Jev (external decision model)	No training data; typed answers with probabilities; ~1/10 the cost of a Haiku call.	Network latency from Saudi Arabia (0.5–1.4 s measured from Asia); vendor figures lack independent tests; judges 'complexity in general', not our models' failures.

Pre-registered keep rule. A router stays only if, on both dev and the shifted set, it cuts cost per successful task by ≥ 5% or p95 latency by ≥ 20% versus the cascade (95% confidence interval excluding zero) without breaching any accuracy floor. Otherwise the loss is reported as a finding.

4. Data design: synthetic, controlled, verifiable

Every ticket is generated, so the right answer is known by construction and difficulty is a dial. One rules file is the single source of truth: it renders the policy document the models read *and* drives the code that computes the correct answers, so the two can never disagree.

Task	Example	Answer format	Cost if wrong
FAQ	“What is the deductible for comprehensive cover?”	one canonical value	low
Intent	Classify a free-text message into one of 8 labels	label	low
Extraction	Pull policy no., date, report no., damage, fault share	5 fields (null if absent)	medium
Adjudication	Decide approve / partial / deny / need-info and the payout	decision + payout (±1 SAR)	high

Split	Size	Arabic share	Purpose
Dev	240	≈ 30% (mostly MSA, light Saudi dialect)	Tuning, floors, router training (cross-validated)
Held-out	240	≈ 30%, same mix	Sealed. Run exactly once at a tagged freeze commit.
Shifted	120	≈ 30%, dialect-leaning	Unseen templates, noise, code-switching; 40 items under changed rules

Difficulty mix (declared, mirrors real queues): L1 FAQ 25% · L2 intent/extraction 30% · L3 one-rule adjudication 20% · L4 multi-rule with distractors 15% · L5 missing or contradictory info 10%. The insurer and its rules are fictional and labelled as such everywhere.

Generation pipeline

- **Sample** case facts per level with a fixed seed → **compute** the answer with the rules engine → **write** text from several template families → optionally **paraphrase** with a cheap model, rejecting any paraphrase that drops or changes a key value → **QA** → freeze files with hashes.
- **QA gates:** 100% schema-valid; zero duplicates or near-duplicates across splits; composition within ±3 points; no template leak; rules-engine unit tests pass; 50 items hand-checked by Hamzah with zero label errors; every failure of the strongest model inspected for generator bugs.

5. Evaluation protocol

The evaluation is designed so that a sceptical judge can rerun it and get the same numbers, and so that no target can be moved after the results are known.

Element	Rule
Success	Correct (deterministic grader) AND total latency ≤ the task's budget.
Cost per successful task	All spend — answers, checks, router calls, retries, failures — ÷ successes.
Accuracy floors (per task)	Best single config's dev accuracy minus 1 point (adjudication) or 2 points (others). Frozen and committed before optimisation.
Latency budgets	p95 latency of that same best config on dev — 'never slower than always-premium'.
Baselines	Always-best · always-cheapest · static rules · cheapest config meeting all floors · naive cascade.
Counterfactual replay	Every config is recorded once per ticket; any policy is then scored by lookup at zero extra cost.
Oracle gap	Cheapest correct config per ticket = the best any router could do; we report how much of that gap we capture.
Statistics	95% paired bootstrap confidence intervals; per-task and per-language breakdowns.
Caching	Headline cost assumes no cache (conservative); modelled cache savings shown separately and labelled.
Held-out	Evaluated once, enforced in code at a tagged commit; simulated vs live gap reported.

6. Robustness and adaptation

- **Shifted set** — unseen phrasings, Saudi dialect, noise and longer messages.
- **Policy change** — 40 tickets under altered rules; the system adapts through configuration only.
- **Scenario switcher** — instantly re-scores the system when accuracy floors tighten, a model's price doubles or a model goes offline, using the recorded data.

7. Quality standards — one charter for every phase

Each standard is a check that either passes or fails. **Blocker** items stop a phase; **Target** items are expected and may only be waived with a logged reason.

Dimension	Level	Standard (checkable)
Evidence	Blocker	Every number traces to a run id, git SHA and config hash. Every improvement is stated against a named baseline with a confidence interval. Simulated and modelled figures are labelled. Negative results are reported.
Data	Blocker	All QA thresholds pass. Answers come only from the rules engine. Held-out never inspected or tuned on. Fiction disclaimer present.
Code	Blocker / Target	Typed contracts at every boundary; lint + tests + replay check + secret scan green before every commit; unit tests for rules engine, graders (incl. Arabic digits), cost accounting, replay, budget guard.
Evaluation	Blocker	Floors, budgets and the router rule committed before the results they judge. Held-out separation enforced in code.
Cost & security	Blocker	Budget guard on every paid run; ledger within 5% of the console. No secret ever committed; a leaked key is revoked immediately.
Reproducibility	Blocker	A fresh clone reproduces every reported number with one command and no API key. Dependencies pinned.
Communication	Target	Every chart titled, with units and source run. Deck has one section per judging criterion. Video 3–4 minutes. Plain language first, technical term second.

Phase gates

Phase (window)	Exit gate
0 Recon & smoke (13:15–14:00)	Repo with secret hygiene committed · prices re-verified · 9-config smoke test logged · Jev reachable, Arabic result logged · mentor booked
1 Data (14:00–17:30)	Rules reviewed · engine tests pass · all QA checks pass · manifest committed · held-out sealed
2 Harness, matrix, floors (17:30–20:00)	Dev matrix recorded · replay reproduces live exactly · solvability checked · baselines generated · floors frozen (tag) · oracle gap · router go/no-go
3 Layers & router (20:30–00:30)	Ablation with CI for every layer · verifier miss rate per task · router keep/drop by rule · chosen policy meets all floors
4 Robustness & freeze (00:30–01:15)	Shifted + policy-change results · ≥ 3 scenarios · prompts frozen · freeze tag
5 Held-out & handoff (01:15–03:00)	Held-out run once · simulated-vs-live gap · report regenerated · fresh-clone test · README · secret scan clean
6 Submission (Saturday)	Deck numbers cross-checked against the report · video public link verified · repo public · project page complete

Review ritual at the end of every phase: run the check command → tick each gate item with evidence → log decisions → commit → three-line status (done / evidence / next). Hard freeze 03:00.

8. Risks and mitigations

Risk	Mitigation
Schedule (≈ 2 h behind plan)	Phase gates, cut order, 03:00 hard freeze
Generator or rules-engine bugs	Unit tests, solvability check, manual review
Template artifacts inflate results	Multiple template families, leak check, unseen families in the shifted set
Effort unsupported on a model	Smoke test first; design works with 6–9 configs
Thinking tokens overspend	Measured in Phase 0; per-run estimate; ask above $5; prepaid cap
Jev access or latency	Local router still competes; access result reported
Cache behaves differently	Headline excludes cache; cache shown as modelled
Small-sample noise	Bootstrap intervals and pre-registered thresholds
Secret leak in the public repo	Pre-commit secret scan, final scan, immediate revocation
Judges cannot run it	Replay mode and fresh-clone test

Cut order under time pressure: router experiment → scenario switcher → audit sampling → bring-your-own-tasks. Never cut: held-out protocol, replay, report, secret hygiene.

9. Locked decisions and open items

- Anthropic models only for answers; 9 configurations; Jev and a local classifier compete as routers.
- Synthetic motor-insurance data; ≈ 30% Arabic (MSA + Saudi dialect); dev 240 / held-out 240 / shifted 120.
- Per-task accuracy floors (global floor as fallback); budgets $100 Anthropic + $5 Jev.
- Deliverables: public GitHub repository, PDF deck, 3–4 minute demo video.

Open: product name · approval of the proposed extract-then-compute variant (a cheap model extracts facts, the rules engine computes the payout) · Jev access confirmation · review of rules wording and intent labels.

Sources

Claude 5.5 pricing: sentra.app/articles/claude-api-pricing; morphllm.com/claude-code-pricing · Effort parameter: platform.claude.com/docs/en/build-with-claude/effort · Jev pricing and latency: aipricing.guru/typesafe-pricing; betonai.net (measured latency from Asia); eesel.ai/blog/typesafe-jev-review · Routing research: RouteLLM (lmsys.org/blog/2024-07-01-routellm); Dekoninck et al., cascade routing (ICML 2025); LLMRouterBench (arxiv.org/abs/2601.07206).